

Thursday — Week 5

Persistence + Retrieval Tuning + Debugging RAG + RAGAS Preview

[ChromaDB Persistence](#) · [top_k Tuning](#) · [Common RAG Failures](#) · [RAGAS Metrics Preview](#)

By the end of today you will be able to:

Persist a ChromaDB collection to disk and reload it without re-embedding documents

Tune top_k and similarity threshold to improve retrieval precision

Diagnose and fix the 5 most common RAG failures using a structured debugging approach

Add metadata to chunks and use ChromaDB where-filters to improve precision

Describe the 4 RAGAS evaluation metrics and what each measures (full implementation in Week 6)

NOTE

Wednesday recap: chunking strategies — fixed-size, recursive (default), semantic, parent-child. Chunk size tuning by document type. split_documents() preserves metadata. Today: production-quality RAG — making it persist, tuning retrieval, and debugging failures.

June 4, 2026

Online (Zoom) · 10:00 AM – 1:00 PM · Mon–Thu | In-Person Friday 10:00 AM – 1:00 PM

Crewlogix — The Institute of Technology · Gulberg III, Lahore

crewlogix

The Institute of Technology

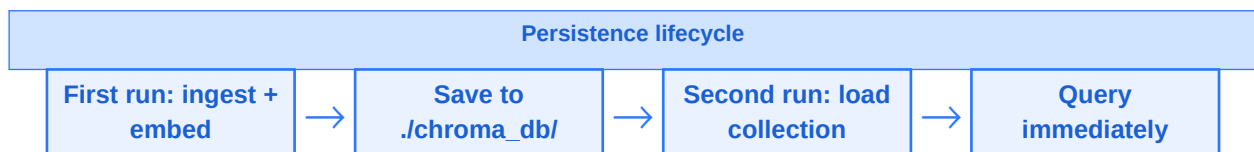
Session — Thursday June 4

3 hours

■ Online — Zoom · 10:00 AM – 1:00 PM

Topic 1 — ChromaDB Persistence: Never Re-embed Twice (30 min)

Re-embedding an entire document collection every time your application starts is wasteful and slow. PersistentClient saves the collection to disk — next run loads instantly with no re-embedding.



1.1 Persist and reload

```

# FIRST RUN: build and save
import chromadb

client = chromadb.PersistentClient(path="./chroma_db")
collection = client.get_or_create_collection(
    name="pakistan_rag", metadata={"hnsw:space": "cosine"})

# ... add documents, embeddings, metadatas ...
print(f"Saved {collection.count()} chunks to disk")

# SECOND RUN (or after Colab restart): just load – no re-embedding
client = chromadb.PersistentClient(path="./chroma_db")
collection = client.get_collection("pakistan_rag")
print(f"Loaded {collection.count()} chunks instantly")
  
```

NOTE

Always use `get_collection()` (not `get_or_create_collection()`) on second run to verify the collection exists. If you use `get_or_create` on an existing collection with the same data, it creates duplicates.

1.2 Check for existing collection before re-ingesting

```

# Smart ingestion: only re-embed if collection is empty or missing
def get_or_build_collection(client, name, documents, emb_model):
    try:
        col = client.get_collection(name)
        if col.count() > 0:
            print(f"Loaded existing collection: {col.count()} chunks")
        return col
    except Exception:
  
```

```
pass # Collection does not exist yet

# Build from scratch
col = client.get_or_create_collection(
    name=name, metadata={"hnsw:space":"cosine"})
embs = emb_model.encode(documents).tolist()
ids = [f"doc_{i}" for i in range(len(documents))]
col.add(ids=ids, documents=documents, embeddings=embs)
print(f"Built new collection: {col.count()} chunks")
return col
```

Topic 2 — Retrieval Tuning (40 min)

Two parameters control the quality of what gets retrieved: **top_k** (how many chunks to return) and **similarity threshold** (minimum score to include). Getting these wrong is the most common cause of RAG quality problems.

2.1 Tuning top_k

top_k value	Effect	When to use
top_k = 1	Single most relevant chunk — very precise but may miss context	Simple, single-fact questions only
top_k = 3	Good balance — enough context without noise — our default	Most use cases
top_k = 5	More context — useful for complex, multi-part questions	Questions requiring synthesis across multiple sections
top_k = 8+	High recall — finds obscure references but may confuse LLM with noise	Last resort — try smaller k with better chunks first

```
# Test different top_k values on the same query
query = "what are the training requirements for PSDF programme?"
for k in [1, 3, 5]:
    results = collection.query(
        query_embeddings=q_emb, n_results=k,
        include=["documents","distances"])
    scores = [round(1-d,3) for d in results["distances"][0]]
    print(f"top_k={k}: scores={scores}")
```

2.2 Similarity threshold filtering

```
# Filter out low-quality retrievals
def retrieve_with_threshold(query, collection, emb_model,
    top_k=5, min_score=0.5):
```

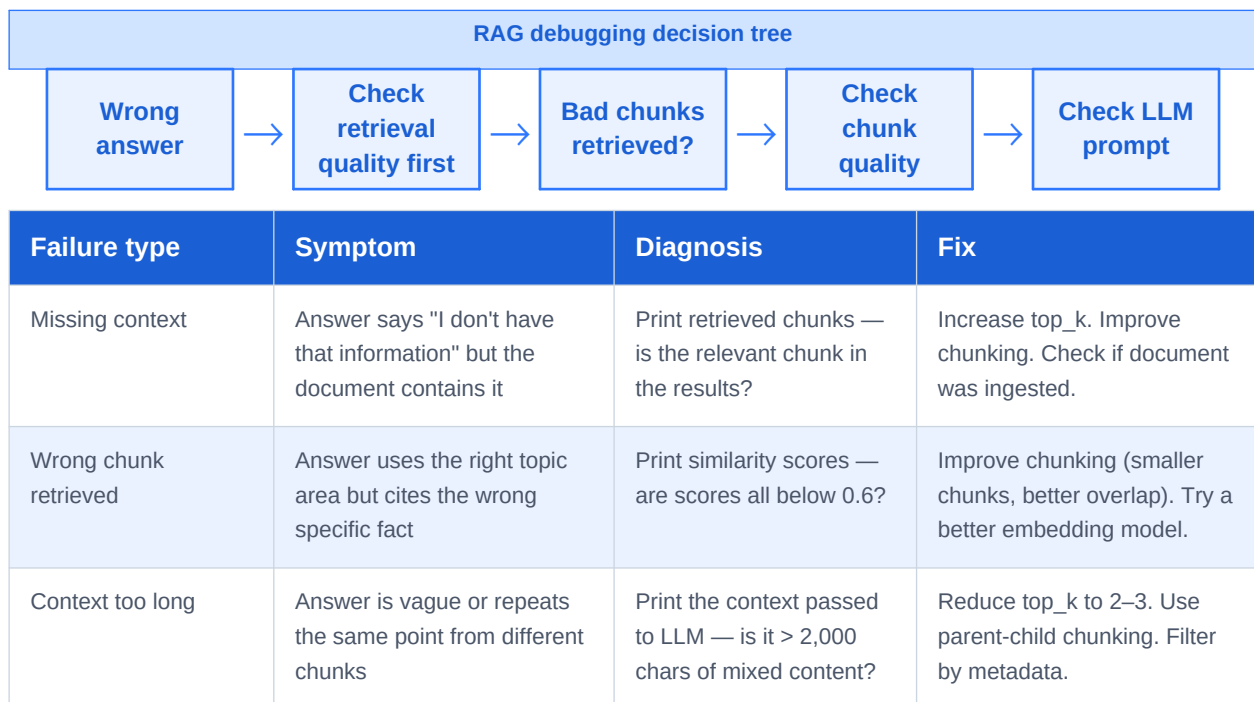
```
q_emb = emb_model.encode([query]).tolist()
results = collection.query(
    query_embeddings = q_emb,
    n_results = top_k,
    include = ["documents", "distances", "metadatas"]
)
filtered = []
for doc, dist, meta in zip(
    results["documents"][0],
    results["distances"][0],
    results["metadatas"][0]):
    score = 1 - dist
    if score >= min_score:
        filtered.append({"doc": doc, "score": score, "meta": meta})
return filtered
```

PAKISTAN

A Pakistani healthcare chatbot with `min_score=0.4` returns drug interaction documents for a question about appointment booking — completely wrong. Raising the threshold to 0.65 prevents hallucinations caused by weakly-relevant context polluting the LLM prompt.

Topic 3 — Debugging RAG Failures (45 min)

When your RAG system gives a wrong or incomplete answer, there is a systematic way to find the problem. Every RAG failure traces back to one of five root causes:



LLM ignores context	Answer contradicts the retrieved chunks	Print the exact prompt — is the context clearly labelled and placed before the question?	Add "Use ONLY the context below. Do not use general knowledge." to the prompt.
Hallucinated answer	Confident answer about something not in any document	Run same query with top_k=10 — are all scores below 0.5?	Add fallback: "If no chunk scores above 0.5, say you don't know."

Debugging toolkit

```
# Step 1: Print exactly what was retrieved
def debug_retrieval(query, collection, emb_model, top_k=5):
    results = retrieve_with_threshold(query, collection, emb_model,
    top_k=top_k, min_score=0.0)
    print(f"\n=== RETRIEVAL DEBUG: {query} ===")
    for i, r in enumerate(results):
        print(f"\n[Chunk {i+1}] Score: {r['score']:.4f}")
        print(f"Source: {r['meta'].get('source', 'unknown')}")
        print(r["doc"][:300])

# Step 2: Print the exact prompt sent to the LLM
def debug_prompt(query, context_chunks):
    context_text = "\n\n---\n\n".join([r["doc"] for r in context_chunks])
    prompt = f"Context:\n{context_text}\n\nQuestion: {query}"
    print(f"\n=== PROMPT ({len(prompt)} chars) ===")
    print(prompt[:1000])
```

Topic 4 — RAGAS Evaluation Preview (25 min)

RAGAS is the standard framework for evaluating RAG pipeline quality automatically. You will implement it fully in Week 6. Today we understand the metrics — so you know what you are optimising for.

RAGAS evaluates all components of the RAG pipeline

Retrieval quality

→

Context precision

→

Context recall

→

Answer quality

→

Faithfulness + Relevance

RAGAS metric	What it measures	Score of 1.0 means	Score of 0.3 means
Faithfulness	Does the answer contain ONLY information from the retrieved context?	Answer is fully grounded — no hallucination	70% of the answer contains hallucinated content not in context
Context Precision	Are the retrieved chunks actually relevant to the question?	All retrieved chunks are relevant — no noise	Most retrieved chunks are off-topic — retrieval is imprecise

Context Recall	Did we retrieve ALL the information needed to answer correctly?	Retrieval found everything needed — nothing important missed	Key information is missing from retrieved chunks
Answer Relevance	Does the answer actually address what was asked?	Answer directly answers the question	Answer is tangentially related but misses the point

EXAM TIP

AWS exam tests RAGAS conceptually: "Which metric measures whether the LLM answer is grounded in the retrieved context?" Answer: Faithfulness. "Which metric measures whether the retriever found all necessary information?" Answer: Context Recall. Both are directly tested.

Exercise — Debug your RAG pipeline (25 min)

1. Take your RAGPipeline from Monday. Run the debug_retrieval() function on 5 queries.

2. For each query, check: what is the highest similarity score? Is it above 0.6?

3. Find one query where the answer is wrong or incomplete. Identify which failure type it is (use the table above).

4. Apply the corresponding fix from the Fix column. Re-run and compare the answer.

Day Summary — Thursday Week 5

Concept	Key practice	Friday lab uses it
Persistence	get_or_build_collection() — only re-embed if empty	Yes — persist the Pakistani document chatbot
top_k tuning	Default 3. Increase for complex Qs. Decrease if context is noisy	Yes — tune during lab
Threshold filter	min_score=0.5–0.65 — prevent irrelevant chunks from reaching LLM	Yes — include in final chatbot
Debug: print retrieval	Always check retrieved chunks before blaming the LLM	Yes — part of the debugging task
RAGAS metrics	Faithfulness / Context Precision / Context Recall / Answer Relevance	Preview — full implementation Week 6

DO THIS

Assignment 1 is due TOMORROW Friday June 5. Submit your 10-use-case prompt engineering portfolio to Moodle → Week 5 → Assignment 1 before the in-person session starts.

